

Отчет по заданию 4
“Теория Параллелизма”
Python Thread

В данном задании требуется реализовать многопоточную версию программы чтения данных с датчиков и формирования картинки, на которой будет видеоизображение вместе с данными, которые были получены из датчиков. Датчики состоят из usb-камеры (SensorCam) и класса SensorX. Для датчика usb-камеры написать класс SensorCam, для остальных датчиков — SensorX. Они должны наследоваться от класса Sensor:

В данной программе была реализована работа считывания и вывода данных из датчиков на основе очередей и потоков. Процесс задержки на получение изображения из очереди реализован задержкой в (1/ fps) секунд, где fps это частота кадров в секунду, остальные датчики считываются без ожидания. Данная работа была предназначена для визуализации того как заставить работать несколько функциональных блоков (датчиков) практически одновременно, а точнее поочередно, прерываясь с 1 потока на другой. Почему почти, потому что интерпретатор Python не умеет работать в физическом многопоточном режиме.

Это было сделано для упрощения автоматизации управления памятью, инициализации и сборки мусора, предотвращения висячих ссылок, внесения больше атомарных операций. Однако такое упрощение негативно сказалось на возможности работы в многопоточном режиме. В итоге для того, чтобы программа смогла выполнять много потоков практически одновременно с минимальными задержками между ними была придумана система GIL- Глобальная блокировка интерпретатора. Она позволяет по очереди за определенный промежуток выполненных байт-кодов (как правило 100) переключать выполнение другого потока. П.С. функции которые могут быть вызваны и прерваны в потоке, для того чтобы закончив другое опять к ним вернутся, называются корутинами.